УНИВЕРЗИТЕТ У НОВОМ САДУ

ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА

DSL за генерисање Spring апликације са Spring Security конфигурацијом

Спецификација пројекта из предмета Доменски оријентисано моделовање и језици

Нови Сад, 2023

Студент:
Јелена Хрњак

Садржај

Увод 1
Концепти апстрактне синтаксе 2
Концепти конкретне синтаксе 4
Закључак 6

Увод

Обезбеђивање одговарајуће заштите за апликације развијене помоћу радног оквира *Spring* сложен је задатак који изискује много времена. Радни оквир *Spring Security* је део радног оквира *Spring* који омогућава додавање безбедносних конфигурација у апликације. Између осталог, овај радни оквир омогућава ауторизацију, аутентификацију и заштиту података на основу различитих елемената као што су улоге, сертификати, корисничко име, лозинка и слично. Да би се наведено обезбедило, потребно је подесити конфигурацију *Spring Security*-ја у апликацији и имплементирати механизме за аутентификацију и контролу приступа, што може бити дуготрајан задатак подложан грешкама, баш због свих ставки на које треба обратити пажњу.

С обзиром да се безбедоносни апекти апликације конфигуришу при креирању сваке апликације, поред тога што је сложен и временски захтеван, овакав посао постаје и репетативан.

Циљ је унапредити креирање безбедних веб апликација тако што би се елиминисали безбедоносни пропусти и грешке, али и уштедело време потребно за имплементацију истих.

Креирањем језика специфичног за домен (*DSL*) намењеног за генерисање *Spring* апликација са подешеном *Spring Security* конфигурацијом у великој мери може да допринесе испуњењу постављеног циља. Овакав језик програмерима, односно доменским експертима, штеди време које би утрошили на иницијализацију апликације и конфигурацију безбедности, а најважније од свега смањује ризик од грешака и безбедносних пропуста.

Концепти апстрактне синтаксе

Пре дефинисања концепата за конфигурацију Spring Security-ја, потребно је навести концепте који су неопходни за генерисање почетне Spring апликације као што су назив и опис саме апликације и детаљи о бази података. Иницијални DSL подржава Postgres, Oracle и MySQL базе података.

Након тога се налази концепти за конфигурацију безбедности у којој је једини подржани механизам JWT (JASON Web Token) те је ову секцију могуће проширити подршком других механизама као што су OAuth и LDAP.

```
application :
    name : string # назив апликације
    description : string # опис апликације

    database : # конфигурација базе података
      type : string # тип базе података (postgres, oracle, mysql)
      url : string # URL адреса базе података
      username : string # корисничко име за приступ бази података
      password : string # лозинка за приступ бази података

    security : # конфигурација безбедности
      mechanism: string # механизам безбедности (тренутно само jwt, прошириво
другим механизмима)
      jwt:
        header:
            signature-algorithm : string # алгоритам за потписивање jwt токена

        payload:
            issuer : string # издавач токена
            expiration-time : integer # време истека токена у секундама
            audience: string # лица овлашћена за коришћење jwt токена
            public-claims : []  # јавне и приватне тврдње које се могу наћи -
            private-claims : [] # - у jwt токенима
        signature:
            secret : string # тајнки кључ за потписивање jwt  токена

    model:
      user: # класа за корисника(подразумевано садржи поља за лозинку и улоге)
        identifier :
          name : string # назив идентификатора (нпр. username, email)
          type : string  # тип идентификатора (нпр. string)
        other-fields : [ # листа осталих поља класе User
          {
              name : string # назив поља
              type : string # тип поља
          }
        ]
```

```
  # могуће проширење за генерисање других класа у моделу
    other: [
      {
        name : string # назив класе
        identifier : # информације о идентификатору
          name : string
          type : string
        other-fields : [  # информације о додатним пољима
          {
              name : string
              type : string
          }
        ]
      }
    ]

  controllers:
    auth: # конфигурација контролера за аутентификацију
      name : string # натив контролера
      path : string # путања
      signup-url : string # ednpoint за регистрацију корисника
      login-url: string # ednpoint за пријаву корисника
      logout-url : string # ednpoint за одјаву корисника
  # могуће проширење за генерисање других контролера
    other: [
        name: string # назив
        path : string # путања
        endpoints: [ # endpoint-ови контролера
            {
              url: string #путања
              method: string # HTTP метод (GET, PUT, POST DELETE)
              roles-authority: [string] #улоге које имају приступ endpoint-у
            }
        ]
      ]
```

Концепте модела и контролера могуће је проширити генерињсањем других класа у моделу и других контролера са endpoint-овима.

Концепти конкретне синтаксе

У даљем тексту налази се пример конкретне синтаксе за апстрактну синтаксу описану у претходном поглављу.

```
application :
    name : "MyDSLGeneratedApp"
    description : "This is an app that is generated with DSL"

    database :
      type : "postgres"
      url : "localhost:5432/mydatabase"
      username : "random_username"
      password : "password123"

    security :
      mechanism: "jwt"
      jwt:
        header:
            signature-algorithm : "HS512"
        payload:
            issuer : "MyApp"
            expiration-time : 1800000
            audience: "AUDIENCE_WEB"
            public-claims : ["email", "username"]
            private-claims : ["role"]
        signature:
            secret : "some_secret"

    model:
      user:
        identifier :
          name : "email"
          type : "string"
        other-fields : [
          {
              name : "username"
              type : "string"
          },
          {
              name : "firstName"
              type : "string"
          },
          {
              name : "lastName"
              type : "string"
          }
        ]
```

```
    other: [
      {
        name : Animal
        identifier :
          name : "id"
          type : "long"
        other-fields : [
          {
              name : "nickname"
              type : "string"
          }
        ]
      }
    ]

  controllers:
    auth:
      name : "AuthController"
      path : "/auth"
      signup-url : "/sign-up"
      login-url: "/login"
      logout-url : "/logout"
    other: [
        name: "UserController"
        path : "/users"
        endpoints: [
            {
              url: "/"
              method: "GET"
              roles-authority: [admin]
            },
            {
              url: "/{id}"
              method: "GET"
              roles-authority: [admin, user]
            }
        ]
      ]
```

Закључак

У спецификацији је описан *DSL* који омогућава генерисање *Spring* апликација које укључују конфигурацију базе података, *Spring Security*-ја и делова модела и контролера. Коришћење овог *DSL*-а програмерима омогућава лакши и бржи почетак развоја *Spring* апликације који представља процес који се понавља при иницијализацији сваког пројекта. Наведене су и могућности проширења описаног *DSL*-а.